



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

WORK INTEGRATED LEARNING PROGRAMMES (WILP) DIVISION

Vidya Vihar, Pilani, Rajasthan – 333031

MTech in Software Engineering

DISSERTATION

Course No.: S2-25_SEZG628T

PerfSense: ML-Based Performance Regression Predictor for React Applications

Abstract & Project Outline Report

Student Name	CHETHAN S
BITS ID	2024TM93208
Degree Program	MTech in Software Engineering
Research Areas	Machine Learning, Performance Engineering, Frontend Development, DevOps
Project Carried Out at	Cadence Design Systems, Bengaluru
Supervisor	Ankit Kagliwal, Sr. Principal Software Engineer
Additional Examiner	Thejaswi Kumar G P, Sr. Principal Software Engineer
Semester	Second Semester, Academic Year 2025-2026
Date	February 2026

1. Broad Area of Work

This dissertation focuses on the intersection of Machine Learning, Performance Engineering, and Modern Web Development. The broad research areas include:

- **Machine Learning in Software Engineering:** Application of predictive models to anticipate software performance issues before deployment.
- **Performance Engineering:** Automated detection and prediction of performance regressions in web applications.
- **Frontend Development & Optimization:** Analysis of React application performance patterns and anti-patterns.
- **DevOps & CI/CD Integration:** Embedding predictive analytics into development workflows for proactive quality assurance.
- **Static Code Analysis:** Extracting performance-relevant features from code changes using Abstract Syntax Tree (AST) parsing.

2. Background

Modern web applications, particularly Single Page Applications (SPAs) built with React, face significant performance challenges that directly impact user experience and business outcomes. Studies indicate that a 1-second delay in page load time can result in a 7% reduction in conversions and 11% fewer page views.

Currently, performance issues in React applications are discovered reactively — after deployment to production when users are already affected. Development teams rely on manual code reviews that may miss performance anti-patterns, post-deployment monitoring tools that alert only after problems occur, and time-consuming manual debugging processes. This reactive approach leads to:

- Degraded user experience
- Increased debugging time (averaging 4–8 hours per performance incident)
- Wasted CI/CD resources
- Potential revenue loss during performance degradation periods

Existing performance monitoring tools (Lighthouse, New Relic, Datadog, Sentry) excel at collecting runtime metrics and visualizing performance trends, but fail to:

- Predict performance regressions before code deployment
- Automatically correlate specific code changes to performance degradation
- Learn from historical patterns to improve predictions over time

There is a critical gap in the industry for a comprehensive system that combines:

- Predictive analysis of code changes before deployment
- Machine learning-based pattern recognition from historical data
- Automated correlation between code commits and performance metrics
- Intelligent, context-aware optimization recommendations

- Seamless integration into the CI/CD pipeline for early detection

3. Objectives

The primary objective of this dissertation is to develop PerfSense, an intelligent performance management platform that shifts performance monitoring from reactive to proactive by predicting regressions before they reach production. The specific objectives are:

- Develop a machine learning model that can accurately predict performance regressions in React applications by analyzing code changes and historical performance metrics before deployment.
- Identify and extract code-level features that are most predictive of performance regressions in React applications through automated static code analysis.
- Design and implement a lightweight monitoring SDK for collecting Core Web Vitals and component-level performance metrics with minimal overhead (<1% performance impact).
- Create an automated feature extraction pipeline that analyzes code commits to identify performance-critical patterns such as new useEffect hooks, Context providers, large components, and missing memoization opportunities.
- Build an end-to-end platform integrating monitoring, prediction, and developer workflow with GitHub webhooks and CI/CD pipeline integration.
- Achieve target model performance metrics: >75% accuracy, >70% precision (to minimize false positives), and >60% recall.
- Evaluate the system through quantitative analysis on 500–1000 labeled commits from open-source React projects and real-world case studies on demo applications.
- Demonstrate that ML-based performance prediction can be practically integrated into real development workflows with minimal friction.

4. Scope of Work

The scope of this dissertation encompasses the complete design, development, and evaluation of PerfSense as an intelligent performance regression prediction system for React applications. The work includes:

- Dataset construction by collecting and labeling 500–1000 commits from 8–10 popular open-source React repositories (React Admin, Ant Design Pro, Grafana UI, Strapi Admin, Metabase frontend, Discourse frontend, Joplin, Sourcegraph web app).
- Development of an automated feature extraction pipeline using TypeScript AST parsing to identify 20–30 performance-relevant features including static code metrics, performance-critical patterns, and historical context.
- Training and evaluation of machine learning models (baseline Logistic Regression and primary Gradient Boosting models using XGBoost/LightGBM) with proper handling of class imbalance and cross-repository validation.
- Implementation of a lightweight React monitoring SDK (<50KB) for collecting Core Web Vitals, component render times, bundle sizes, and resource loading metrics.
- Development of backend API service (Node.js/Express or Python/FastAPI) with PostgreSQL database for metrics ingestion, project management, and analytics.

- Creation of Python-based ML inference microservice for real-time regression predictions with structured response including probability scores, risk levels, contributing factors, and recommendations.
- Development of web-based dashboard using React and TypeScript with views for commit analysis, project overview, real-time monitoring, and integration management.
- Integration with GitHub webhooks for automated analysis on new commits and pull request status checks.
- Comprehensive evaluation through quantitative metrics, real-world case studies with deliberate performance regressions, and system performance benchmarking.

The dissertation excludes production deployment to commercial environments, support for frameworks other than React, and comprehensive security hardening (though basic authentication will be implemented). The focus remains on proof-of-concept demonstration and academic validation of the predictive approach.

 **GitHub Repository:** <https://github.com/mach2605/PerfSense>

5. Test Application Specifications

5.1 Training & Validation Dataset (Primary)

The system will be trained and validated on 8–10 open-source React applications of varying sizes to ensure the ML model learns patterns that generalize across different codebase scales:

Small Applications (10–30 pages/routes) — 2–3 Applications

- Joplin (note-taking app): ~20 routes, 15,000 LOC
- Similar complexity applications from GitHub

Medium Applications (30–100 pages/routes) — 3–4 Applications

- Strapi Admin: ~50 routes, 80,000 LOC
- Discourse frontend: ~60 routes, 120,000 LOC
- Similar complexity applications

Large Applications (100+ pages/routes) — 2–3 Applications

- Grafana UI: ~150+ routes, 500,000+ LOC
- Ant Design Pro: ~100+ routes, 200,000 LOC
- Metabase frontend: ~80 routes, 300,000 LOC

Total Dataset Coverage

Metric	Target Range
Unique pages/routes	500–800 across all applications
Lines of code per application	10,000 – 500,000+ LOC
Component count per application	50 – 1,000+ components

Bundle size per application	500 KB – 5 MB+
Labeled commits analyzed	500–1,000 commits

5.2 Real-World Case Studies (Proof of Concept)

2–3 demo applications will be built specifically for controlled testing with deliberate performance regressions:

Demo App 1: E-Commerce Application (~15 Pages)

Home, Product List, Product Detail, Cart, Checkout, User Profile, Orders, Wishlist, Payment, Order Confirmation, Search Results, Categories, Reviews, Account Settings, Contact.

Demo App 2: Dashboard Application (~20 Pages)

Dashboard Home, Analytics Overview, Real-time Metrics, Historical Reports, User Management, Role Management, Settings, Notifications, Profile, API Keys, Logs, System Health, Performance Monitor, Data Export, Import Tools, Audit Trail, Integrations, Webhooks, Documentation, Support.

Demo App 3: Content Management System (~25 Pages)

Posts List, Post Editor (Rich Text), Media Library, Image Gallery, Categories Management, Tags Management, Comments Moderation, User Profiles, SEO Settings, Site Settings, Theme Customization, Plugin Management, Analytics Dashboard, Backup/Restore, Database Manager, API Documentation, Permissions, Workflow, Publishing Queue, Archive, Search Console.

Total demo applications: ~60 pages across 3 applications.

5.3 Rationale for Application Selection

- Diverse dataset — Applications of different sizes ensure generalization.
- Real-world representation — Open-source apps reflect actual production complexity.
- Controlled testing — Demo apps allow deliberate regression introduction.
- Statistical significance — 500–1,000 commits provide sufficient ML training data.

6. React Anti-Patterns Detection Framework

The system will detect and analyze 20+ React-specific anti-patterns that cause performance regressions, categorized into 6 main groups:

6.1 Category 1: Component Rendering Issues

1. Unnecessary Re-renders

Problem: Components without proper memoization causing excessive re-renders.

```
// Anti-pattern
```

```
function ExpensiveList({ items, onClick }) {  
  return items.map(item => <Item key={item.id} onClick={onClick} />;  
}
```

Detection Criteria:

- Large lists (>50 items) without React.memo()
- Components receiving new function/object references on every render
- Missing useMemo/useCallback for expensive computations

2. Inline Function Definitions in JSX

Problem: New function instances created on every render.

```
// Anti-pattern  
<button onClick={() => handleClick(item.id)}>Click</button>
```

Detection Criteria:

- Arrow functions directly in JSX props
- Occurs in lists or frequently re-rendering components

3. Nested Component Definitions

Problem: Component defined inside another component causing complete remount.

```
// Anti-pattern  
function Parent() {  
  function Child() { return <div>Child</div>; }  
  return <Child />;  
}
```

Detection Criteria: Function component defined inside another component's body.

4. Large Component Files

Problem: Single component with excessive complexity.

Detection Criteria:

- Single component file >300 lines
- 10+ useState hooks in one component
- Complex nested JSX (>5 levels deep)

6.2 Category 2: State Management Issues

5. Excessive useState Hooks

Problem: Too many individual state variables instead of grouped state.

Detection Criteria:

- 8+ useState hooks in a single component
- Related state not grouped using useReducer

6. Missing Dependency Arrays in useEffect

Problem: Missing or incorrect dependencies causing stale closures.

```
// Anti-pattern  
useEffect(() => {  
  fetchData(userId);  
}, []); // userId not in dependencies!
```

Detection Criteria:

- useEffect with missing dependencies
- Variables used inside effect not listed in dependency array

7. Context Provider Without Memoization

Problem: Context value changes on every render causing all consumers to re-render.

```
// Anti-pattern
<MyContext.Provider value={{ user, theme, settings }}>
```

Detection Criteria:

- Context provider with object/array value not wrapped in useMemo
- Context provider high in component tree with multiple consumers

8. Synchronous State Updates in Loops

Problem: Multiple setState calls causing multiple re-renders.

```
// Anti-pattern
items.forEach(item => {
  setState(prevState => [...prevState, item]);
});
```

Detection Criteria: setState inside forEach/map/for loop without batching.

6.3 Category 3: Bundle Size & Code Splitting Issues

9. Missing Code Splitting / Lazy Loading

Problem: All routes imported eagerly, increasing initial bundle size.

Detection Criteria:

- No React.lazy() usage for route components
- Large bundle size (>1MB for initial load)

10. Heavy Dependencies Imported Unnecessarily

Problem: Entire library imported for one function.

```
// Anti-pattern
import _ from 'lodash'; // 500KB for one function
```

Detection Criteria:

- Large library imports (moment.js, lodash) without tree-shaking
- Package.json additions with large bundle impact

11. Large Media/Assets Not Optimized

Problem: Large images loaded eagerly without optimization.

Detection Criteria:

- Image files >500KB added to repository
- No lazy loading for images below the fold
- Missing modern formats (WebP, AVIF)

12. No Bundle Analysis

Problem: Bundle size increases without investigation.

Detection Criteria:

- Bundle size increases >10% in a single commit
- Duplicate dependencies in bundle

6.4 Category 4: Data Fetching & Side Effects

13. Fetching Data in Render

Problem: API calls running on every render instead of in `useEffect`.

```
// Anti-pattern
function Component() {
  const data = fetchData(); // Runs on every render!
  return <div>{data}</div>;
}
```

Detection Criteria:

- Fetch/axios calls outside `useEffect`
- API calls in render phase

14. No Request Deduplication

Problem: Multiple components fetching same data simultaneously.

Detection Criteria:

- Same API endpoint called multiple times in parallel
- No caching mechanism (React Query, SWR)

15. Infinite Re-render Loops

Problem: State update causing the effect to run infinitely.

```
// Anti-pattern
useEffect(() => {
  setState(value + 1);
}, [value]); // Runs forever!
```

Detection Criteria:

- `useEffect` updates state that is in its dependency array
- Circular dependencies detected

6.5 Category 5: Memory Leaks & Cleanup

16. Missing Cleanup in `useEffect`

Problem: Subscriptions/timers continue after component unmount.

```
// Anti-pattern
useEffect(() => {
  const subscription = subscribe();
  // No return function to cleanup!
}, []);
```

Detection Criteria: `useEffect` with subscriptions/timers/event listeners without cleanup return function.

7. Plan of Work

Phase	Work to be Done
Month 1 Foundation & Data Preparation	Week 1–2: Literature Review - Comprehensive review of performance monitoring tools, ML in software engineering, and React optimization techniques - Produce refined problem statement and evaluation framework (15–20 pages) Week 3–4: Data Collection - Collect data from 8–10 open-source React repositories, run Lighthouse CI on historical commits, label regression commits - Target: 500–1,000 labeled commits with exploratory data analysis
Month 2 Feature Engineering & Model Development	Week 1–2: Feature Extraction Pipeline - Implement TypeScript AST parser, extract static code features and React-specific patterns - Feature documentation covering 20–30 features Week 3–4: ML Model Development - Develop baseline (Logistic Regression) and primary models (XGBoost/LightGBM), handle class imbalance, hyperparameter tuning - Target: >75% accuracy, >70% precision. Produce trained models and evaluation report
Month 3 System Implementation	Week 1: Monitoring SDK - Build React/TypeScript monitoring SDK for Web Vitals collection (<50KB, <1% overhead). NPM-ready package with documentation Week 2: Backend API - Develop REST API (Node.js/Express or FastAPI), PostgreSQL database, GitHub webhook integration. Functioning API with documentation Week 3: ML Inference Microservice - Create Python ML inference microservice for real-time predictions (<2 seconds latency). Containerized service integrated with backend Week 4: Dashboard UI - Build React dashboard with commit analysis, performance visualization, real-time metrics. Responsive web interface with user documentation
Month 4 Evaluation & Documentation	Week 1: System Evaluation - System evaluation on held-out commits, real-world case studies (2–3 demo applications), measure prediction accuracy vs baselines - Produce comprehensive evaluation report with statistical analysis Week 2: Refinements

- Refinements based on evaluation findings, fix critical bugs, improve model accuracy if needed, additional experiments
- Production-quality system with final model

Week 3–4: Dissertation Writing

- Write 80–100 pages covering: Introduction, Literature Review, System Design, Implementation, Evaluation, Conclusions
- Final dissertation document ready for review and submission

8. Literature References

The following are preliminary literature references that will guide the research. Additional references on machine learning applications in software engineering, performance prediction models, static code analysis techniques, and React performance optimization best practices will be included during the literature review phase.

- [1] N. Nikiforou, I. Malavolta, and P. Lago, "A Taxonomy of Software Performance Issues in Web Development," Proceedings of the International Conference on Software Analysis, Evolution and Reengineering (SANER), 2019.
- [2] H. Zhang, "An Investigation of the Relationships between Lines of Code and Defects," IEEE International Conference on Software Maintenance, 2009.
- [3] M. Greiler, A. van Deursen, and M. A. Storey, "Automated Detection of Test Fixture Strategies and Smells," International Conference on Software Testing, Verification and Validation, 2013.
- [4] J. Walden, M. Doyle, G. A. Welch, and M. Whelan, "Security of Open Source Web Applications," Proceedings of the 3rd International Symposium on Empirical Software Engineering and Measurement, 2009.
- [5] A. Chowdhury and M. Hindle, "GreenMiner: A Hardware Based Mining Software Repositories Software Energy Consumption Framework," Proceedings of the 13th Working Conference on Mining Software Repositories, 2016.
- [6] React Team, "Optimizing Performance," React Documentation, 2023. [Online]. Available: <https://react.dev/reference/react/optimization>
- [7] Google Developers, "Web Vitals," web.dev, 2023. [Online]. Available: <https://web.dev/vitals/>
- [8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
- [9] D. Amalfitano et al., "Using GUI Ripping for Automated Testing of Android Applications," Proceedings of the IEEE/ACM International Conference on Automated Software Engineering, 2012.
- [10] S. McConnell, Code Complete, 2nd ed. Redmond, WA: Microsoft Press, 2004.

9. Particulars of the Supervisor and Examiner

	Supervisor	Additional Examiner
Name	Ankit Kagliwal	Thejaswi Kumar G P

Qualification	MTech	MTech
Designation	Senior Principal Software Engineer	Senior Principal Software Engineer
Organization & Location	Cadence Design Systems Pvt. Ltd.RMZ Ecoworld Rd, Adarsh Palm Retreat,Bellandur, Bengaluru, Karnataka 560103	Cadence Design Systems Pvt. Ltd.RMZ Ecoworld Rd, Adarsh Palm Retreat,Bellandur, Bengaluru, Karnataka 560103
Phone No.	+91 80 4184 5768	
Email Address	kagliwal@cadence.com	thejaswi@cadence.com

10. Remarks of the Supervisor

This dissertation proposal presents a well-structured and timely research project addressing a critical gap in modern web application development. The proposed PerfSense system combines machine learning with performance engineering in an innovative way that has significant practical and academic value.

The student has demonstrated comprehensive understanding of the problem domain, having thoroughly reviewed existing performance monitoring tools and their limitations. The research questions are clearly formulated and appropriately scoped for an MTech dissertation. The proposal shows strong alignment with the student's specialization in Full Stack Engineering and effectively integrates concepts from multiple coursework subjects including Artificial and Computational Intelligence, DevOps, API-based Products, Software Quality and Assurance, and Cloud Computing.

The technical approach is sound, with a well-defined system architecture that combines monitoring, feature extraction, machine learning prediction, and developer workflow integration. The choice of technologies (React, TypeScript, Python, XGBoost/LightGBM, PostgreSQL) is appropriate and demonstrates practical awareness of industry-standard tools. The proposed evaluation methodology is rigorous, including both quantitative metrics (accuracy, precision, recall) and qualitative case studies.

The 4-month timeline is realistic and shows careful planning with clear milestones and deliverables. The student has identified appropriate open-source React repositories for dataset construction and has set achievable target performance metrics (>75% accuracy, >70% precision).

This project addresses a real industry problem and has potential for both academic contribution (novel predictive approach to performance regression detection) and practical impact (reducing time to detect issues, preventing revenue loss, improving developer productivity). The work could lead to publishable research and open-source contributions.

The student has shown enthusiasm, technical competence, and thorough preparation during our discussions. I am confident in their ability to execute this research plan successfully and deliver a high-quality dissertation that meets the standards expected for an MTech degree in Software Engineering.

I approve this dissertation proposal and look forward to supervising this promising research project.

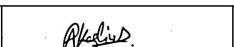
Dissertation Outline — Summary

DISSERTATION OUTLINE — S2-25_SEZG628T	
Student ID No.	2024TM93208
Name of Student	CHETHAN S
Student's Email Address	chethans@cadence.com
Student's Employing Organization & Location	Cadence Design Systems Pvt. Ltd., RMZ Ecoworld Rd, Adarsh Palm Retreat, Bellandur, Bengaluru, Karnataka 560103
Supervisor's Name	Ankit Kagliwal
Supervisor's Employing Organization & Location	Cadence Design Systems Pvt. Ltd., RMZ Ecoworld Rd, Adarsh Palm Retreat, Bellandur, Bengaluru, Karnataka 560103
Supervisor's Email Address	kagliwal@cadence.com
Additional Examiner's Name	Thejaswi Kumar G P
Additional Examiner's Employing Organization & Location	Cadence Design Systems Pvt. Ltd., RMZ Ecoworld Rd, Adarsh Palm Retreat, Bellandur, Bengaluru, Karnataka 560103
Additional Examiner's Email Address	thejaswi@cadence.com
Dissertation / Project Work Title	PerfSense: ML-Based Performance Regression Predictor for React Applications

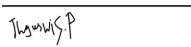
Signatures



Student — CHETHAN S



Supervisor — Ankit Kagliwal



Additional Examiner —
Thejaswi Kumar G P